3D-LUTによる GamutMapping

2021/09/10

2025/09/16

金森克洋

やりたいこと

- Gamut-Mapping処理を３D-LUTで実現
- 処理はYCrCb空間でのSaturation_UPを実装
- GamutMappingの非線形変換処理で補間結果に影響が出来るのかを確認
- 環境はPython3.8.10＋openCV4.4.0

処理方法

C:¥Users¥kanamori¥VSC_py38¥S_UP1.py

S_UP1.py
make_Sup_3dlut(GRID_NUM, sat_up , out_gamut_proc)

save_3dlut(lut3d, GRID_NUM, filename):

3D-LUTファイル（.cube形式）

C:¥Users¥kanamori¥VSC_py38¥make_3dlut.py

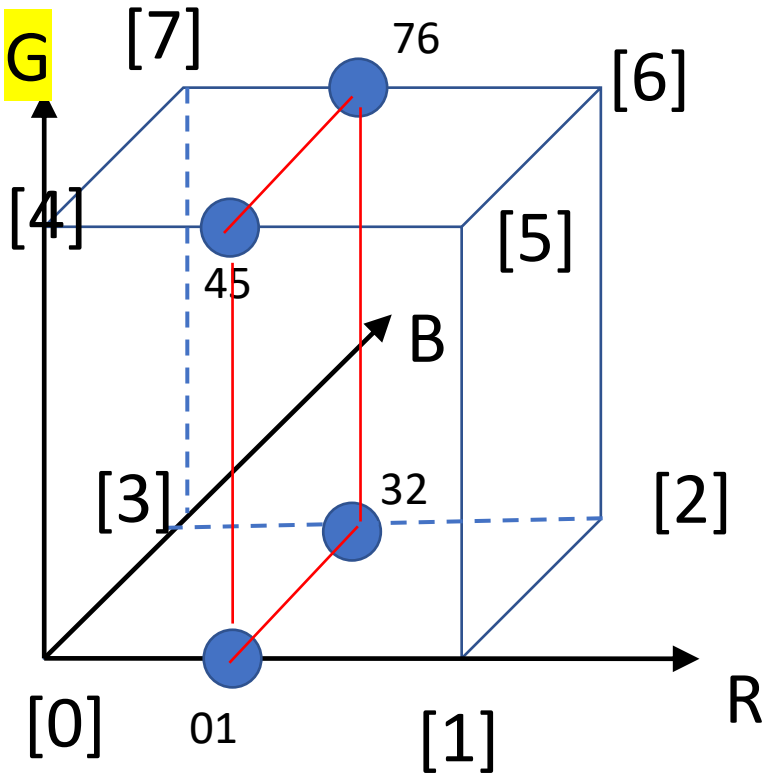
RGB画像

Make_3dlut.py

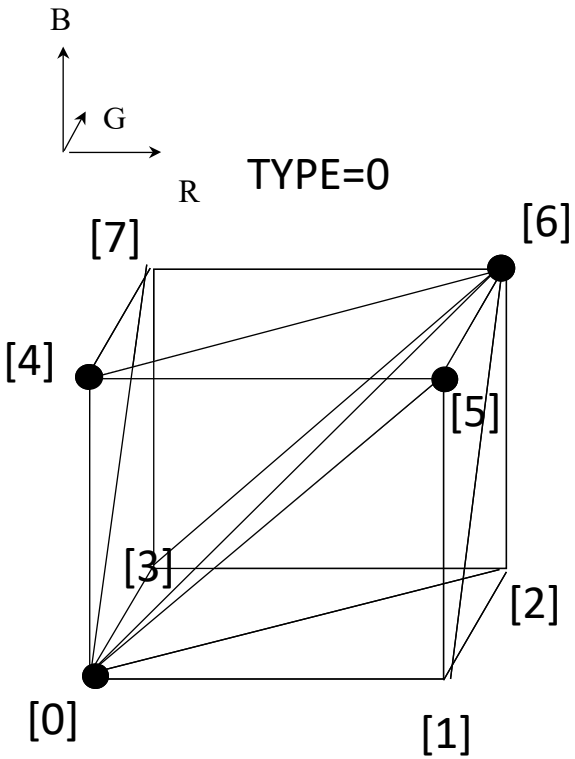
read_3dlut(filename)
inter_cube(b_in, g_in, r_in, lut):

RGB画像

inter_cube補間

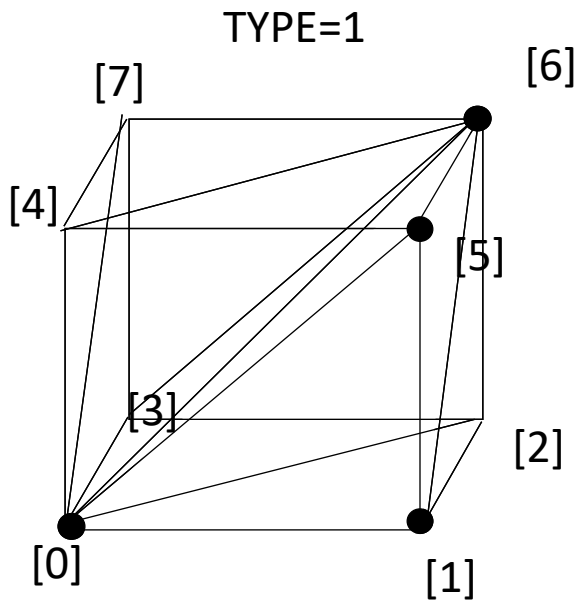


inter_tetra補間



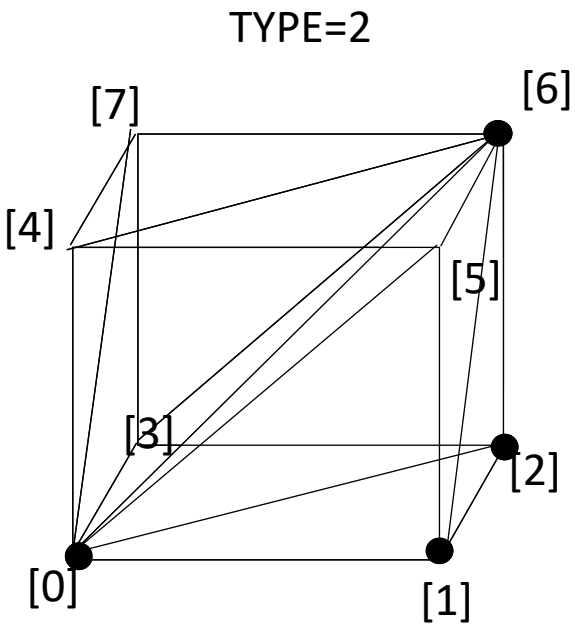
R > G
B > R
B > G

∴B > R > G



R > G
B < R
B > G

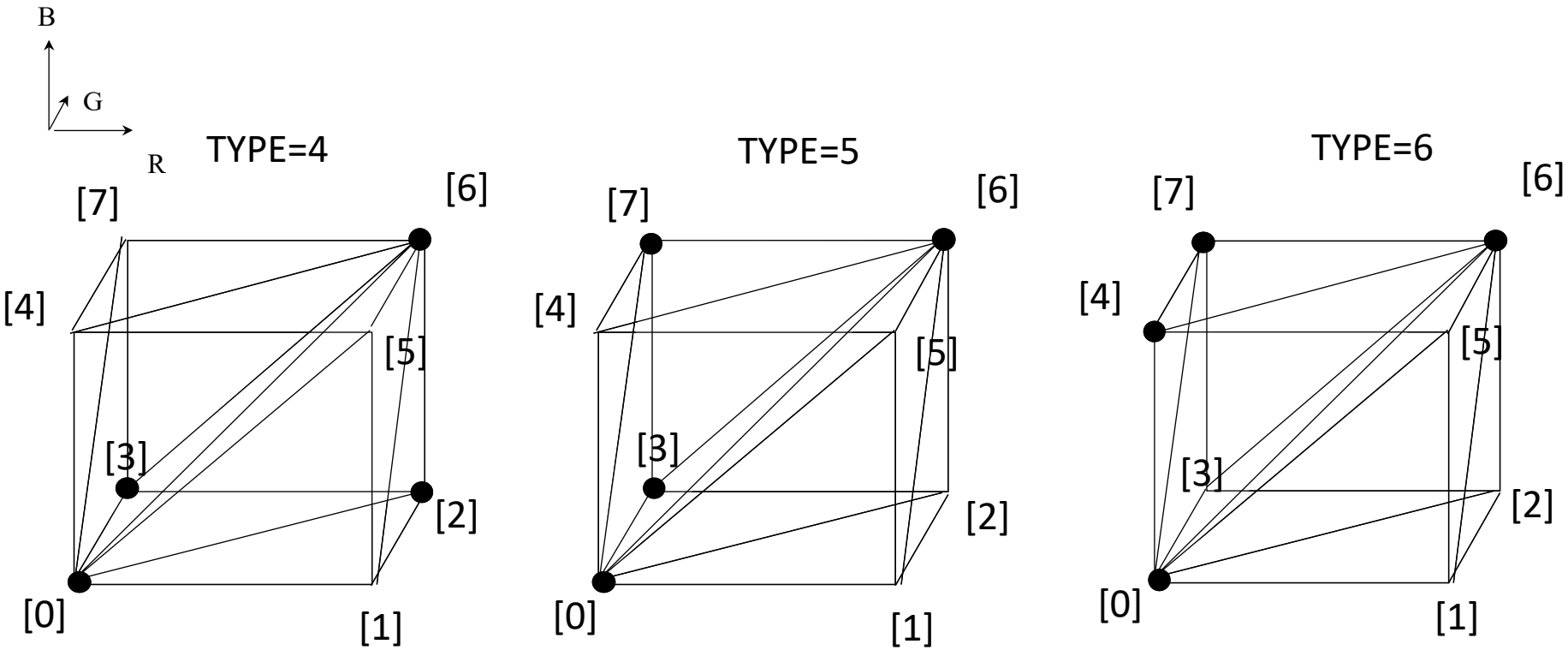
∴R > B > G



R > G
B < R
B < G

∴R > G > B

inter_tetra補間



R<G
B<R
B<G

∴G > R > B

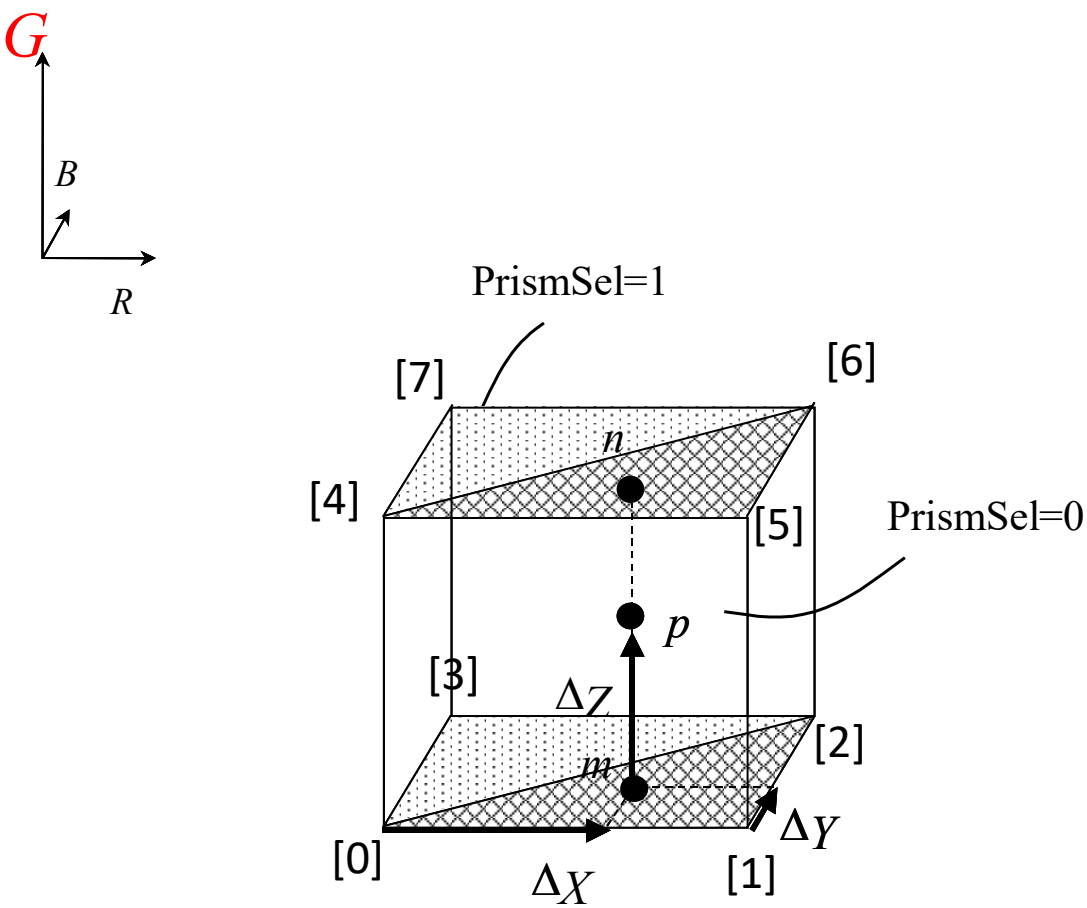
R<G
B>R
B<G

∴G > B > R

R<G
B>R
B>G

∴B > G > R

prism補間



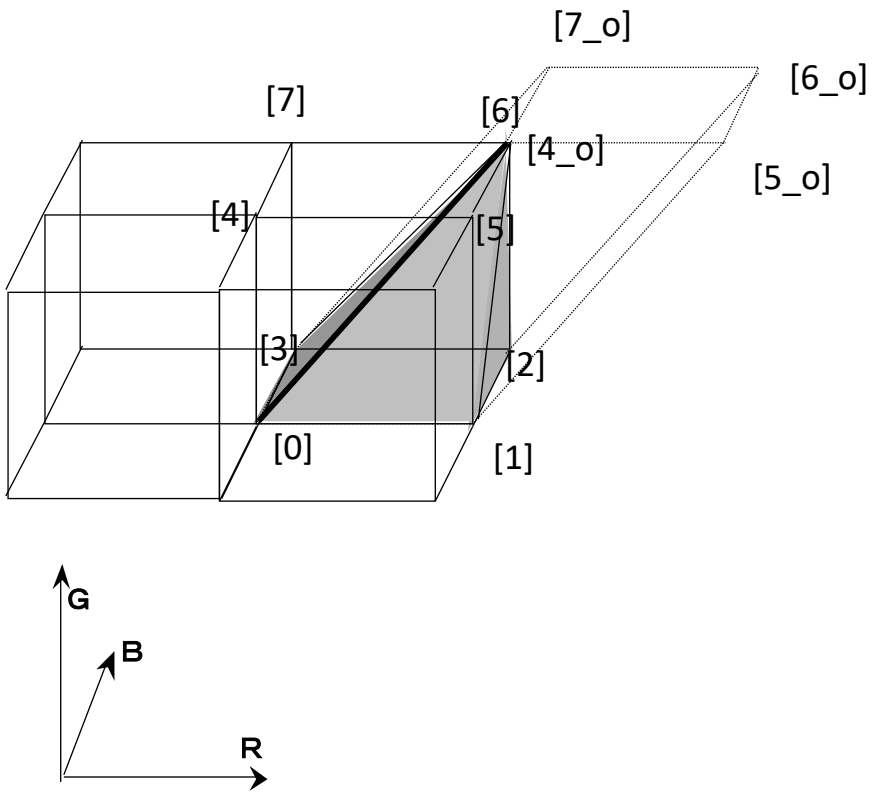
slant-prism補間

Base番号（0）

R_lower>G_lower &&
B_lower>G_lower

上底ずらし量
u_rb=+1
u_bb=+1

重み係数補正
Kr=R_lower-G_lower
Kb=B_lower-G_lower



PrismSel(KR,KB使用）
0:KR>=KB
1:KR<KB

Vertex	
[0]	
[1]	
[2]	
[3]	
[4]	[4_o]
[5]	[5_o]
[6]	[6_o]
[7]	[7_o]

slant-prism補間

Base番号(1)

R>G && B<G

上底ずらし量

u_RB=+1 u_BB =0

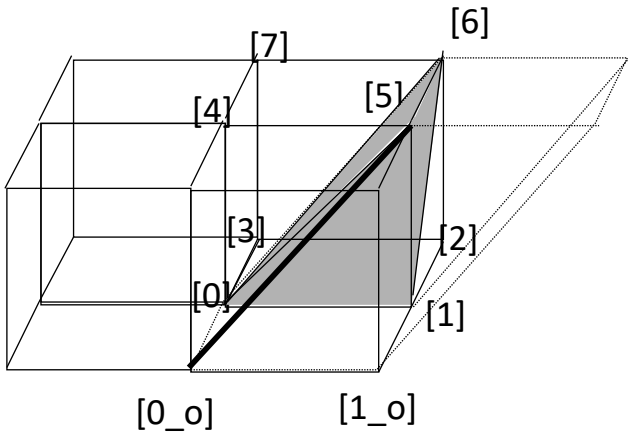
下底ずらし量

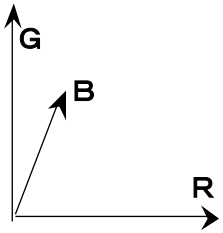
l_Rb=0 l_bb=-1

重み係数

KR=R-G

KB=D + B-G





PrismSel

1:KR<KB

Vertex	
[0]	
[1]	
[2]	
[3]	
[4]	
[5]	
[6]	
[7]	

slant-prism補間

Base番号(2)

R<G && B>G

上底ずらし量

u_Rb=0　　u_BB=+1

下底ずらし量

l_Rb=-1　　l_bb=0

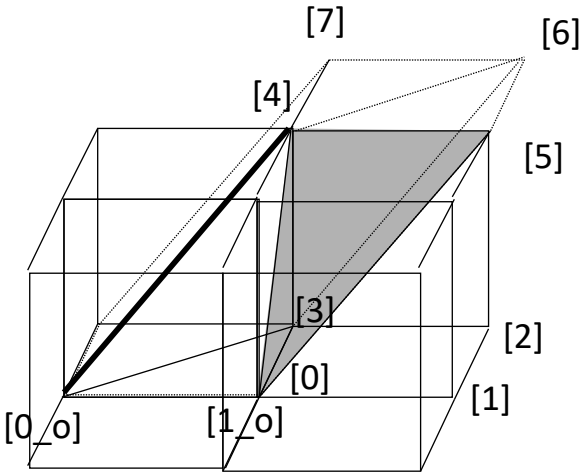
重み係数

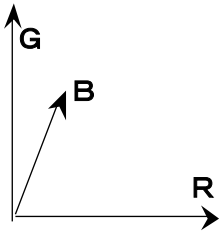
KR=D +R-G

KB=B-G

PrismSel

0:KR>KB





Vertex	
[0]	
[1]	
[2]	
[3]	
[4]	
[5]	
[6]	
[7]	

slant-prism補間

Base番号(3)

R<G && B<G

下底ずらし量

l_Rb=-1　　l_bb=-1

重み係数

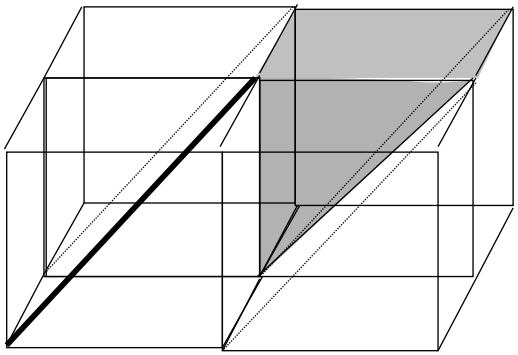
KR=D + R-G

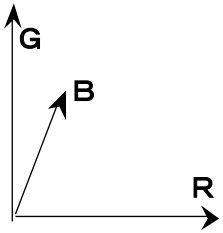
KB=D + B-G

PrismSel

0:KR>=KB

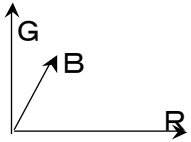
1:KR<KB

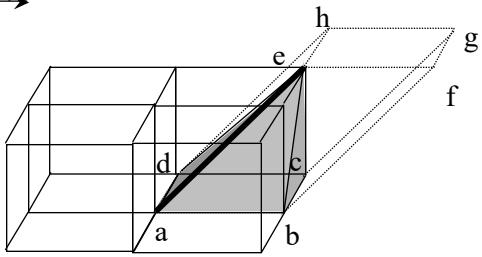




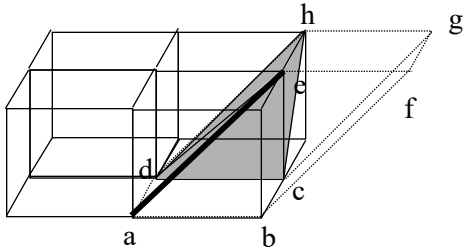
Vertex	
[0]	[0_o]
[1]	[1_o]
[2]	[2_o]
[3]	[3_o]
[4]	
[5]	
[6]	
[7]	

slant-prism補間

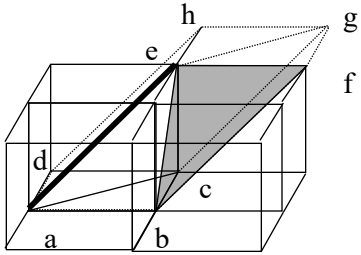




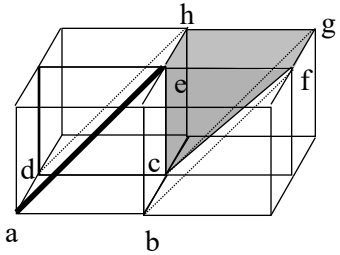
Base番号（0）



Base番号（1）

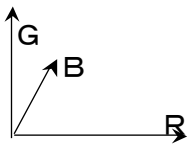


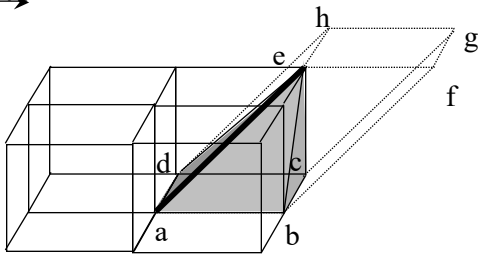
Bsae番号（2）



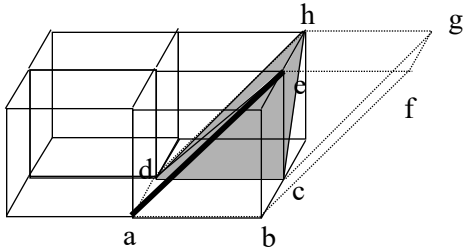
Base番号（3）

slant-cube補間

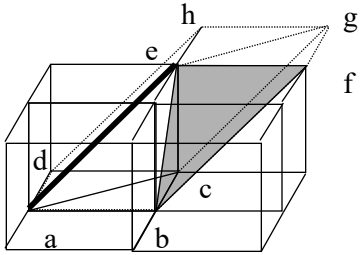




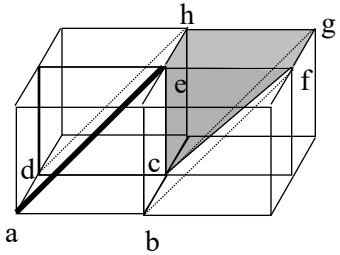
Base番号（0）



Base番号（1）

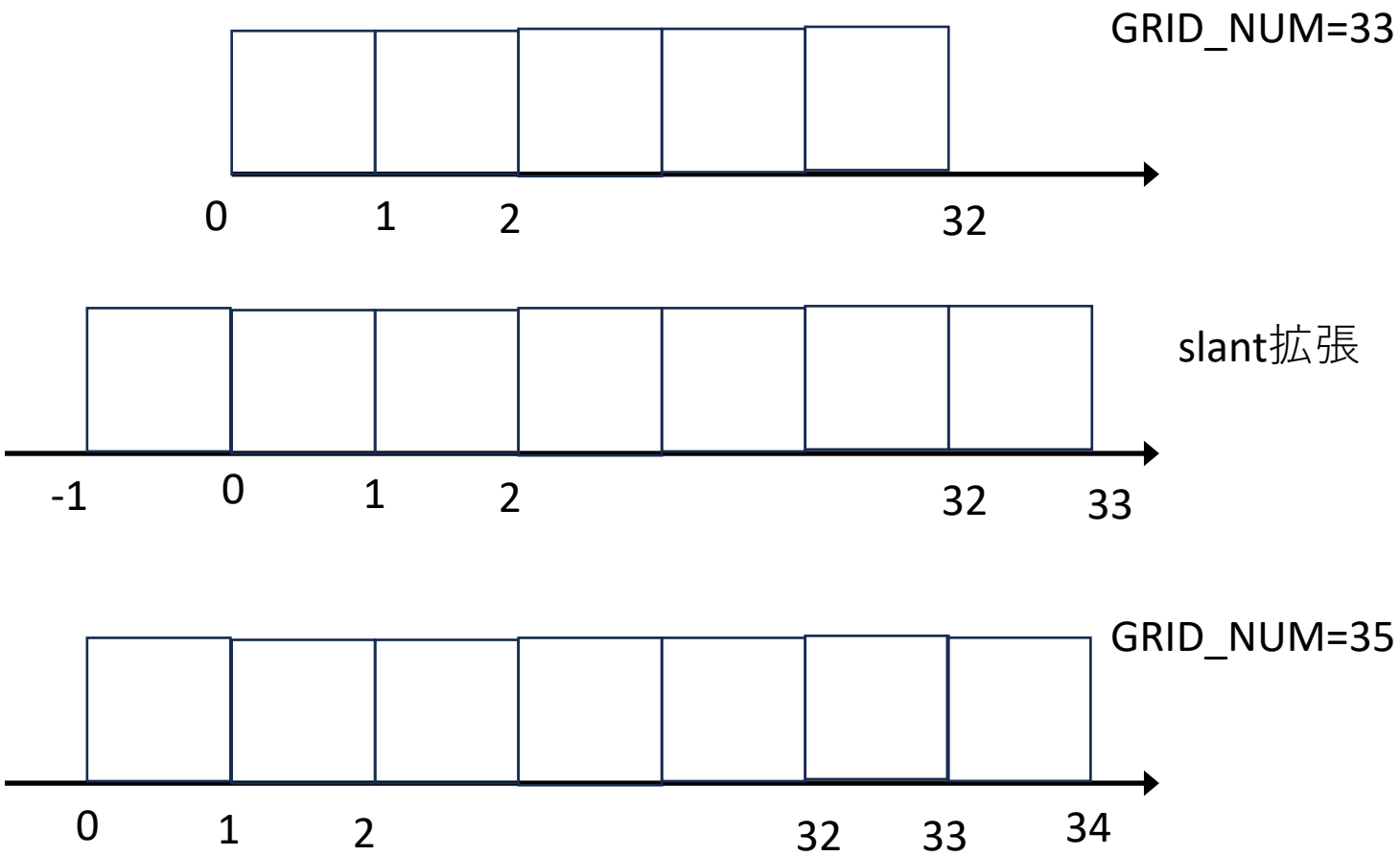


Bsae番号（2）

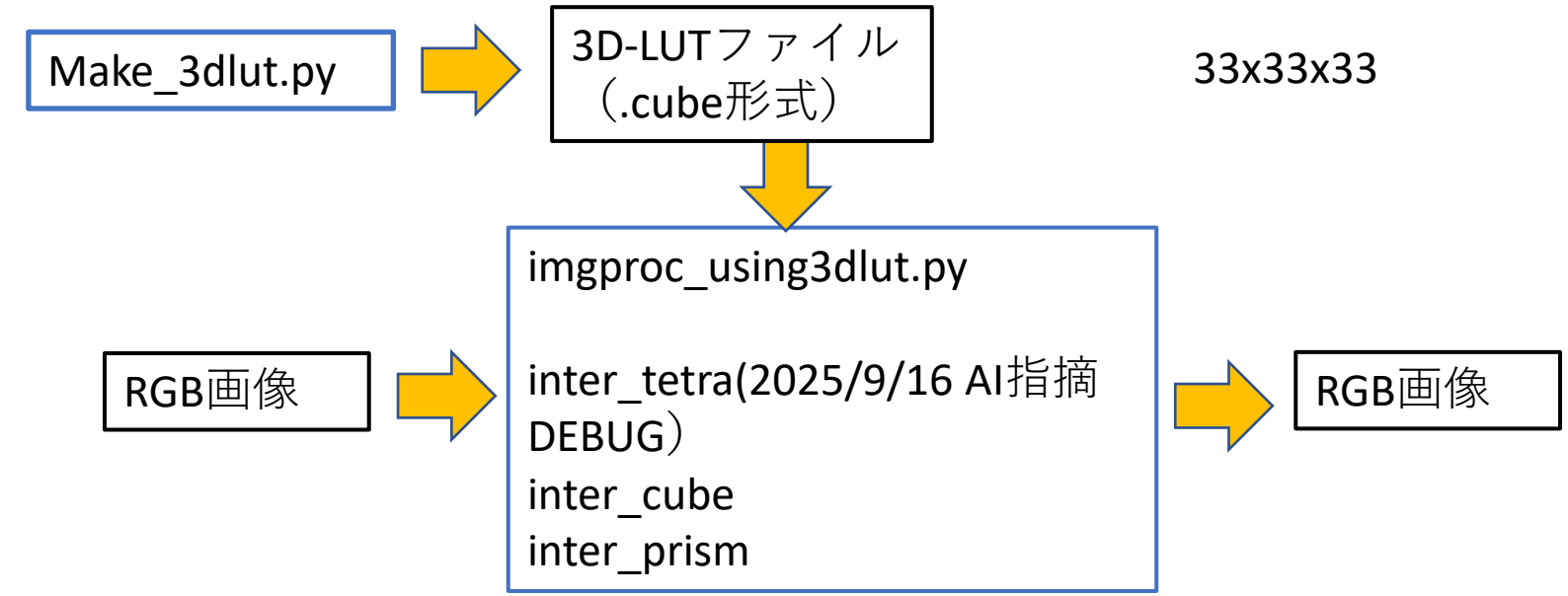


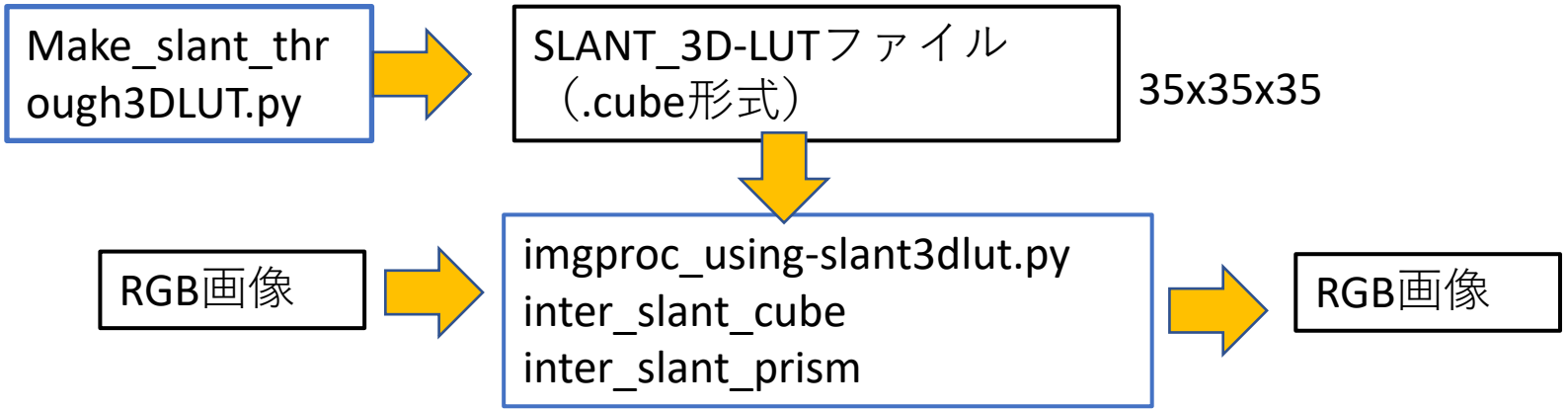
Base番号（3）

slant-prism用3D-LUT



処理方法


Make_3dlut.py
3D-LUTファイル
（.cube形式）
33x33x33
imgproc_using3dlut.py
inter_tetra(2025/9/16 AI指摘
DEBUG）
inter_cube
inter_prism
RGB画像
RGB画像


Make_slant_thr
ough3DLUT.py
SLANT_3D-LUTファイル
（.cube形式）
35x35x35
imgproc_using-slant3dlut.py
inter_slant_cube
inter_slant_prism
RGB画像
RGB画像

Cube LUT specification (Adobe)

Cube LUT Specification

Version 1.0

The example function is {r, (3 * g + b)/4.0, b}.

The example uses the default domain 0 1.

Note that the first component (the red channel) changes fastest.

```
# Created Tue 19 Mar 2013
LUT_3D_SIZE 2
0 0 0
1 0 0
0 .75 0
1 .75 0
0 .25 1
1 .25 1
0 1 1
1 1 1
```

処理結果（LUT33x33x33)

X3S_CLIPPING
CALICULATION

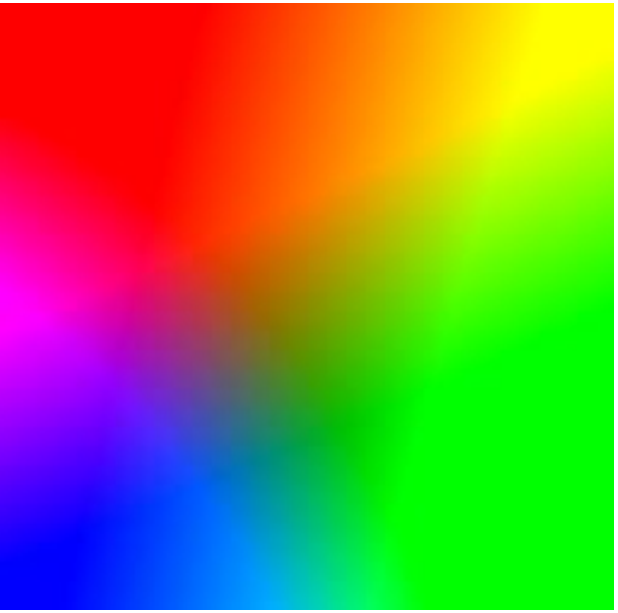
X3S_CLIPPING
3D-LUT

処理結果（LUT33x33x33)

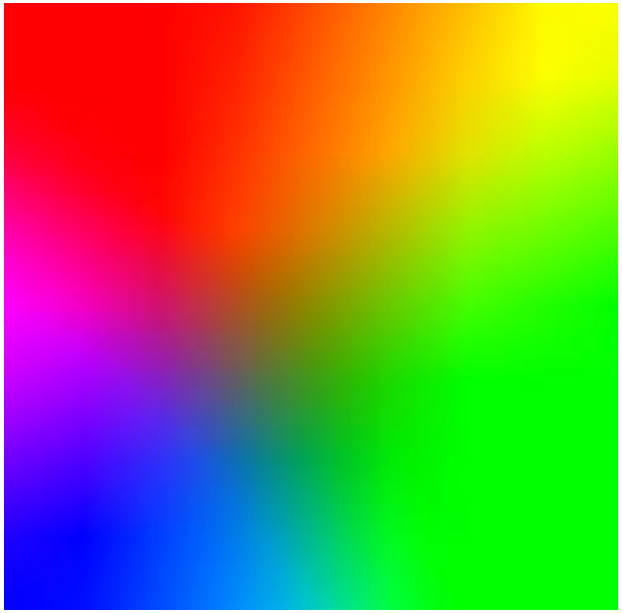
X3S_GAMUT-MAPPING
CALICULATION

X3S_GAMUT-MAPPING
3D-LUT

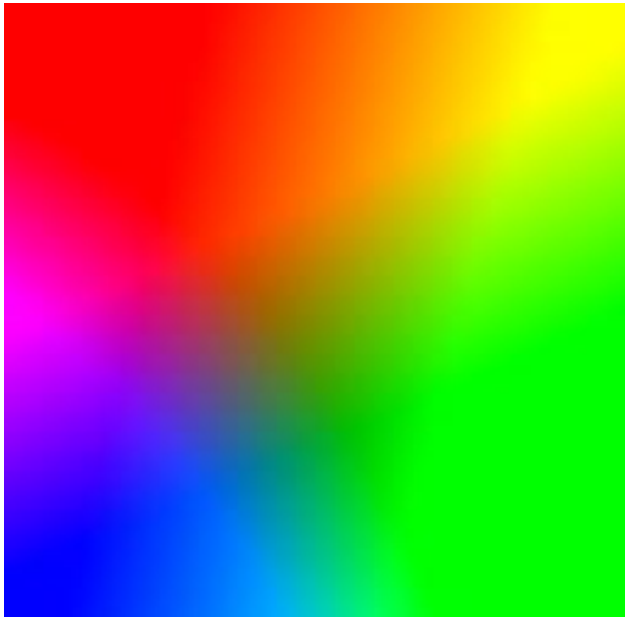
処理結果（LUT9x9x9 vs 33x33x33)



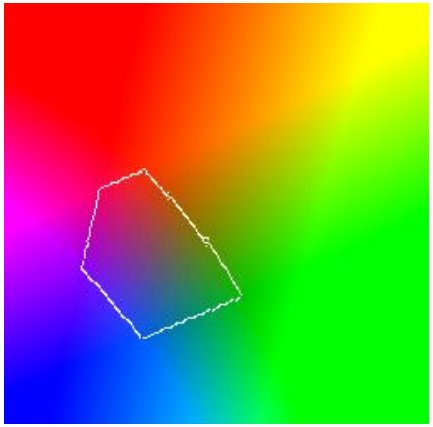
X3S_CLIPPING
CALICULATION



X3S_CLIPPING
3D-LUT(9x9x9)

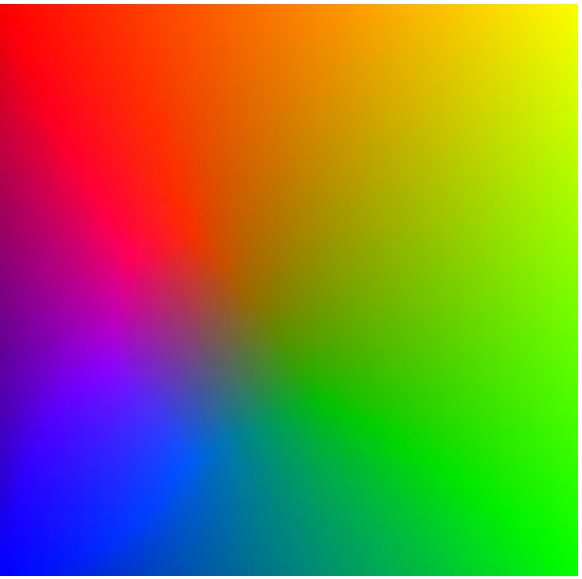


X3S_CLIPPING
3D-LUT(33x33x33)

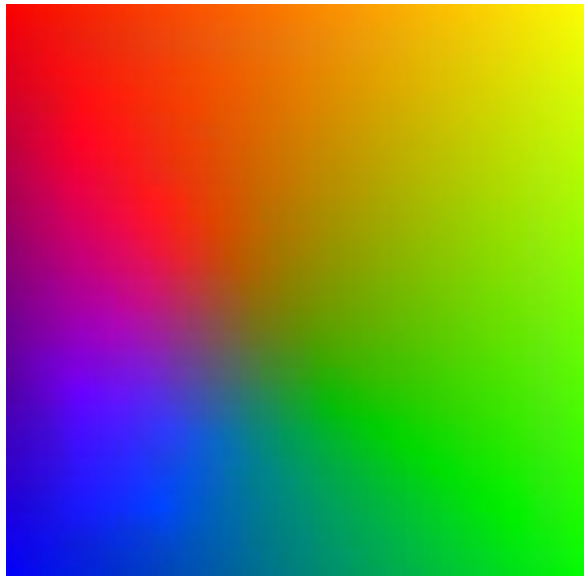


■不明点
・３D-LUT(9x9x9)のアーチファクトは
Gamut境界で発生している訳ではない

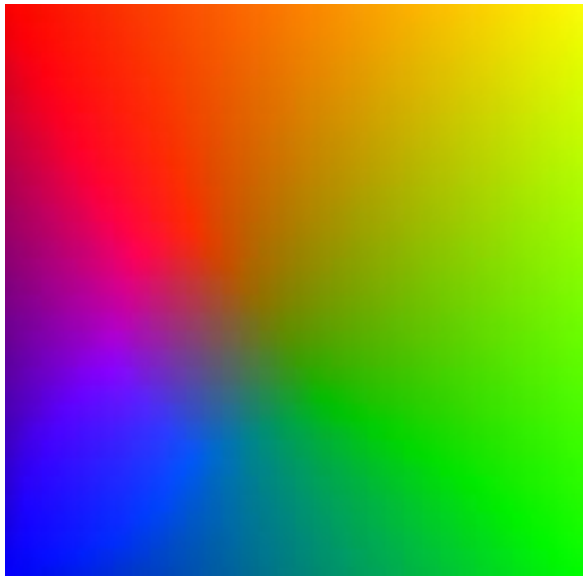
処理結果（LUT9x9x9 vs 33x33x33)



X3S_MAPPING
CALICULATION



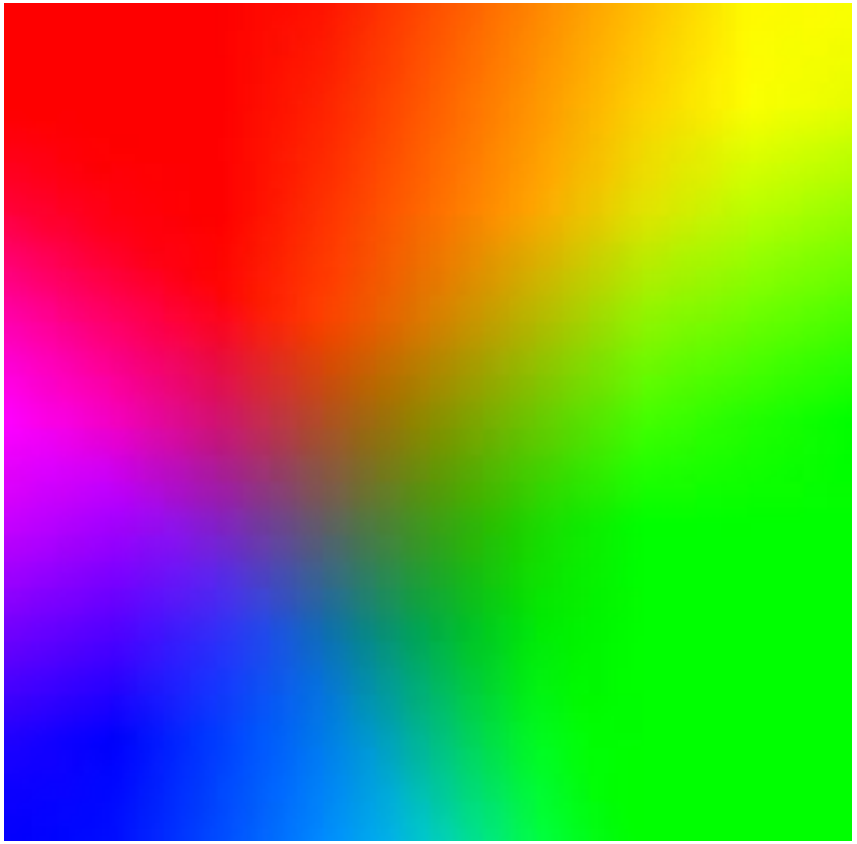
X3S_MAPPING
3D-LUT(9x9x9)



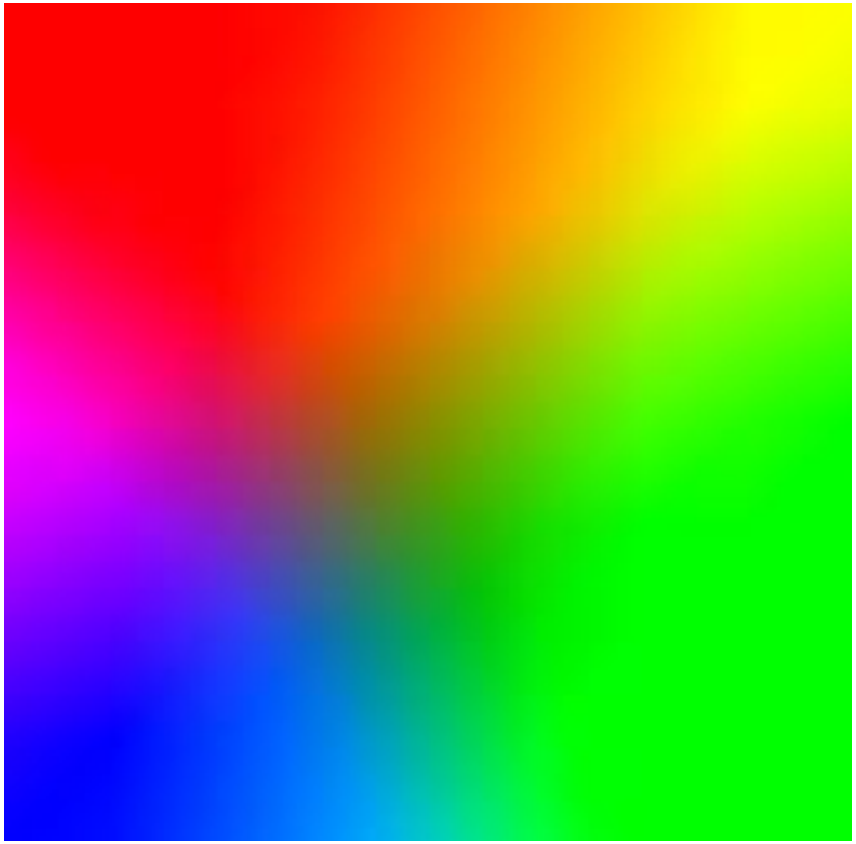
X3S_MAPPING
3D-LUT(33x33x33)

処理結果（cubic vs tetrahedral)

■X3S_CLIPPING　3D-LUT(9x9x9)



CUBIC (Tri-Lineear）
INTERPOLATION



TETRAHEDRAL
INTERPOLATION